ast ::= var | (**App** ast ast . . .) | val
val ::= (**Fun** var ast) | $atom$
| (**List** val . . .) | stx | (Defs σ)
$atom$ ::= sym | $prim$ | $tprim$ | ....
sym ::= /var/
$prim$ ::= `stx-e` | `mk-stx` | ....
$tprim$ ::= `new-defs` | `def-bind` | `value` | `lexpand`
σ ::= $addr$ | NULL

stx ::= (**Stx** $atom$ ctx)
| (**Stx** (**List** stx . . .) ctx)
id ::= (**Stx** sym ctx)
ctx ::= •
| (Rename ctx id var σ)
| (Mark ctx mrk)
| (Defs ctx σ)

ξ ::= expand-time environment, $sym \to transformer$
Σ ::= definition-context store, $\sigma \to (id \to sym)$
S ::= set of σ

E ::= [ ] | (**App** val . . . E ast . . .)
F ::= [ ]
| (**Stx** (**List** stx . . . F ⟨Exp stx ξ⟩ . . .) ctx)
| ⟨Eval E[⟨Local F σ⟩] ξ $mode$⟩

$mixture$ ::= ⟨Exp stx ξ⟩
| ⟨Eval $local\text{-}ast$ ξ $mode$⟩
| (**Stx** (**List** stx . . . $mixture$ . . .) ctx)
$local\text{-}ast$ ::= ast | E[⟨Local $mixture$ σ⟩]
$mode$ ::= ⟨InExp mrk⟩
| ⟨InBind id stx⟩
| ⟨InDef σ id E mrk⟩

$transformer$ ::= Fun | Let-Syntax | Quote
| (Var id) | val | Stop

var, $addr$, mrk ::= a name

Figure 1: Grammar

⟨F[⟨Exp (**Stx** (**List** id_{lam} id_{arg} stx_{body}) ctx) ξ⟩], Σ⟩ [lambda]
⟶ ⟨F[(**Stx** (**List** id_{lam} id_{new} ⟨Exp $stx_{newbody}$ ξ_{new}⟩) ctx)], Σ⟩
where Fun = ξ(resolve[[id_{lam}, Σ]]), id_{new} = rename[[id_{arg}, id_{arg}, var_{new}]],
$stx_{newbody}$ = rename[[stx_{body}, id_{arg}, var_{new}]], ξ_{new} = ξ+{$var_{new} \to$(Var id_{new})},
var_{new} fresh

⟨F[⟨Exp (**Stx** (**List** id_{rator} stx_{arg} . . .) ctx) ξ⟩], Σ⟩ [app-var]
⟶ ⟨F[(**Stx** (**List** ⟨Exp id_{rator} ξ⟩ ⟨Exp stx_{arg} ξ⟩ . . .) ctx)], Σ⟩
where (Var id_{new}) = ξ(resolve[[id_{rator}, Σ]])

⟨F[⟨Exp (**Stx** (**List** (**Stx** (**List** stx_1 . . .) ctx_1) stx_2 . . .) ctx_2) ξ⟩], Σ⟩ [app-exp]
⟶ ⟨F[(**Stx** (**List** ⟨Exp (**Stx** (**List** stx_1 . . .) ctx_1) ξ⟩ ⟨Exp stx_2 ξ⟩ . . .) ctx_2)], Σ⟩

⟨F[⟨Exp id ξ⟩], Σ⟩ ⟶ ⟨F[id_{new}], Σ⟩ [ref]
where (Var id_{new}) = ξ(resolve[[id, Σ]])

⟨F[⟨Exp (**Stx** (**List** id_{quote} stx) ctx) ξ⟩], Σ⟩ [quote]
⟶ ⟨F[(**Stx** (**List** id_{quote} stx) ctx)], Σ⟩
where Quote = ξ(resolve[[id_{quote}, Σ]])

Figure 2: Parsing reductions

$\langle F[\langle \text{EVAL } E[ast_{expand}]\ \xi\ mode\rangle], \Sigma\rangle$ [local-start]
$\longrightarrow \langle F[\langle \text{EVAL } E[\langle \text{LOCAL } \langle \text{EXP } stx_{new}\ \xi_{stops}\rangle\ \sigma\rangle]\ \xi\ mode\rangle], \Sigma\rangle$
where (**APP lexpand** stx_{expr} (**LIST** id_{stop} . . .) (DEFS σ)) = ast_{expand},
$\langle \text{INEXP } mrk\rangle = mode$,
ξ_{stops} = nostops[[ξ]]+{resolve[[id_{stop}, Σ]]$\rightarrow$STOP} . . . ,
stx_{new} = defs[[mark[[stx_{expr}, mrk]], σ]]

$\langle F[\langle \text{EVAL } E[\langle \text{LOCAL } stx\ \sigma\rangle]\ \xi\ mode\rangle], \Sigma\rangle$ [local-done]
$\longrightarrow \langle F[\langle \text{EVAL } E[\text{defs[[mark[[}stx, mrk\text{]], }\sigma\text{]]}]\ \xi\ mode\rangle], \Sigma\rangle$
where $\langle \text{INEXP } mrk\rangle = mode$

$\langle F[\langle \text{EXP } (\textbf{STX } (\textbf{LIST } id_{stop}\ stx\ \ldots)\ ctx)\ \xi\rangle], \Sigma\rangle$ [stop]
$\longrightarrow \langle F[(\textbf{STX } (\textbf{LIST } id_{stop}\ stx\ \ldots)\ ctx)], \Sigma\rangle$
where STOP = ξ(resolve[[id_{stop}, Σ]])

$\langle F[\langle \text{EXP } (\textbf{STX } (\textbf{LIST } id_{ls}\ id_{mac}\ stx_{rhs}\ stx_{body})\ ctx)\ \xi\rangle], \Sigma\rangle$ [bind-start]
$\longrightarrow \langle F[\langle \text{EVAL parse[[}stx_{rhs}, \Sigma\text{]] }\xi\ \langle \text{INBIND } id_{mac}\ stx_{body}\rangle\rangle], \Sigma\rangle$
where LET-SYNTAX = ξ(resolve[[id_{ls}, Σ]])

$\langle F[\langle \text{EVAL } val\ \xi\ \langle \text{INBIND } id_{mac}\ stx_{body}\rangle\rangle], \Sigma\rangle$ [bind-done]
$\longrightarrow \langle F[\langle \text{EXP rename[[}stx_{body}, id_{mac}, var_{new}\text{]] }\xi\text{+}\{var_{new}\rightarrow val\}\rangle], \Sigma\rangle$
where var_{new} fresh

$\langle F[\langle \text{EXP } stx_{macapp}\ \xi\rangle], \Sigma\rangle$ [app-macro]
$\longrightarrow \langle F[\langle \text{EVAL } (\textbf{APP } val\ \text{mark[[}stx_{macapp}, mrk_{new}\text{]]})\ \xi\ \langle \text{INEXP } mrk_{new}\rangle\rangle], \Sigma\rangle$
where (**STX** (**LIST** $id_{mac}\ stx_{arg}$ . . .) ctx) = stx_{macapp},
val = ξ(resolve[[id_{mac}, Σ]]), mrk_{new} fresh

$\langle F[\langle \text{EVAL } stx_{expmac}\ \xi\ \langle \text{INEXP } mrk_{new}\rangle\rangle], \Sigma\rangle$ [macro-done]
$\longrightarrow \langle F[\langle \text{EXP mark[[}stx_{expmac}, mrk_{new}\text{]] }\xi\rangle], \Sigma\rangle$

$\langle F[\langle \text{EVAL } E[(\textbf{APP } (\textbf{FUN } var\ ast)\ val)]\ \xi\ mode\rangle], \Sigma\rangle$ [eval-lam]
$\longrightarrow \langle F[\langle \text{EVAL } E[ast[var\leftarrow val]]\ \xi\ mode\rangle], \Sigma\rangle$

$\langle F[\langle \text{EVAL } E[(\textbf{APP } prim\ val_{arg}\ \ldots)]\ \xi\ mode\rangle], \Sigma\rangle$ [eval-prim]
$\longrightarrow \langle F[\langle \text{EVAL } E[val_{res}]\ \xi\ mode\rangle], \Sigma\rangle$
where val_{res} = δ($prim\ val_{arg}$ . . .)

$\langle F[\langle \text{EVAL } E[(\textbf{APP value } id)]\ \xi\ \langle \text{INEXP } mrk\rangle\rangle], \Sigma\rangle$ [eval-local]
$\longrightarrow \langle F[\langle \text{EVAL } E[val]\ \xi\ \langle \text{INEXP } mrk\rangle\rangle], \Sigma\rangle$
where val = ξ(resolve[[mark[[id, mrk]], Σ]])

$\langle F[\langle \text{EVAL } E[(\textbf{APP new-defs})]\ \xi\ mode\rangle], \Sigma\rangle$ [new-defs]
$\longrightarrow \langle F[\langle \text{EVAL } E[(\text{DEFS } \sigma)]\ \xi\ mode\rangle], \Sigma\text{+}\{\sigma\rightarrow\emptyset\}\rangle$
where σ fresh

$\langle F[\langle \text{EVAL } E[(\textbf{APP def-bind } val_{def}\ id)]\ \xi\ \langle \text{INEXP } mrk\rangle\rangle], \Sigma\rangle$ [def-var]
$\longrightarrow \langle F[\langle \text{EVAL } E[0]\ \xi\text{+}\{var_{new}\rightarrow(\text{VAR } id_{new})\}\ \langle \text{INEXP } mrk\rangle\rangle], \Sigma_2\rangle$
where val_{def} = (DEFS σ), id_{new} = rename[[id, id, var_{new}]],
Σ_2 = Σ+{$\sigma\rightarrow\Sigma(\sigma)$+{mark[[$id$, mrk]]$\rightarrow var_{new}$}},
var_{new} fresh

$\langle F[\langle \text{EVAL } E[(\textbf{APP def-bind } val_{def}\ id\ stx_{expr})]\ \xi\ mode\rangle], \Sigma\rangle$ [def-start]
$\longrightarrow \langle F[\langle \text{EVAL parse[[}stx_{newexpr}, \Sigma\text{]] }\xi\ \langle \text{INDEF } \sigma\ id\ E\ mrk\rangle\rangle], \Sigma\rangle$
where val_{def} = (DEFS σ), $\langle \text{INEXP } mrk\rangle = mode$,
$stx_{newexpr}$ = mark[[stx_{expr}, mrk]]

$\langle F[\langle \text{EVAL } val\ \xi\ \langle \text{INDEF } \sigma\ id\ E\ mrk\rangle\rangle], \Sigma\rangle$ [def-done]
$\longrightarrow \langle F[\langle \text{EVAL } E[0]\ \xi\text{+}\{var_{new}\rightarrow val\}\ \langle \text{INEXP } mrk\rangle\rangle], \Sigma_2\rangle$
where Σ_2 = Σ+{$\sigma\rightarrow\Sigma(\sigma)$+{mark[[$id$, mrk]]$\rightarrow var_{new}$}},
var_{new} fresh

Figure 3: Transformer reductions

δ(**`stx-e`** (**Stx** *val ctx*)) = *val*
δ(**`mk-stx`** *atom* (**Stx** *val ctx*)) = (**Stx** *atom ctx*)
δ(**`mk-stx`** (**List** *stx* . . .) (**Stx** *val ctx*)) = (**Stx** (**List** *stx* . . .) *ctx*)
...

parse[[(**Stx** (**List** id_{lambda} id_{arg} stx_{body}) *ctx*), Σ]] = (**Fun** resolve[[id_{arg}, Σ]] parse[[stx_{body}, Σ]])
 where `lambda` = resolve[[id_{lambda}, Σ]]
parse[[(**Stx** (**List** id_{quote} *stx*) *ctx*), Σ]] = strip[[*stx*]]
 where `quote` = resolve[[id_{quote}, Σ]]
parse[[(**Stx** (**List** id_{syntax} *stx*) *ctx*), Σ]] = *stx*
 where `syntax` = resolve[[id_{syntax}, Σ]]
parse[[(**Stx** (**List** stx_{rator} stx_{rand} . . .) *ctx*), Σ]] = (**App** parse[[stx_{rator}, Σ]] parse[[stx_{rand}, Σ]] . . .)
parse[[*id*, Σ]] = resolve[[*id*, Σ]]

resolve[[*stx*, Σ]] = resolve*[[*stx*, Σ, $\emptyset$, $\emptyset$]]

resolve*[[(**Stx** /*var*/ •), Σ, S_{spine}, S_{branch}]] = *var*
resolve*[[(**Stx** /*var*/ (Mark *ctx mrk*)), Σ, S_{spine}, S_{branch}]] = resolve*[[(**Stx** /*var*/ *ctx*), Σ, S_{spine}, S_{branch}]]
resolve*[[(**Stx** /*var*/ (Rename *ctx* id_{orig} var_{new} σ)), Σ, S_{spine}, S_{branch}]] = var_{new}
 where var_1 = resolve*[[id_{orig}, Σ, S_{branch}, S_{branch}]], var_2 = resolve*[[(**Stx** /*var*/ *ctx*), Σ, {σ}$\cup S_{spine}$, S_{branch}]],
 $var_1 = var_2$, marksof[[id_{orig}]] = marksof[[(**Stx** /*var*/ *ctx*)]]
resolve*[[(**Stx** /*var*/ (Rename *ctx* id_{orig} var_2 σ)), Σ, S_{spine}, S_{branch}]] = resolve*[[(**Stx** /*var*/ *ctx*), Σ, S_{spine}, S_{branch}]]
resolve*[[(**Stx** /*var*/ (Defs *ctx* σ)), Σ, S_{spine}, S_{branch}]] = resolve*[[(**Stx** /*var*/ *ctx*), Σ, S_{spine}, S_{branch}]]
 where $\sigma \in S_{spine}$
resolve*[[(**Stx** /*var*/ (Defs *ctx* σ)), Σ, S_{spine}, S_{branch}]] = resolve*[[(**Stx** /*var*/ ctx_{new}), Σ, S_{spine}, {σ}$\cup S_{branch}$]]
 where +{$id \rightarrow var_{new}$} . . . = $\Sigma(\sigma)$, ctx_{new} = renames[[σ, ((id var_{new}) . . .), *ctx*]]

renames[[σ, (), *ctx*]] = *ctx*
renames[[σ, ((*id var*) (id_2 var_2) . . .), *ctx*]] = renames[[σ, ((id_2 var_2) . . .), (Rename *ctx id var* σ)]]

mark[[(**Stx** *atom ctx*), *mrk*]] = (**Stx** *atom* (Mark *ctx mrk*))
mark[[(**Stx** (**List** *stx* . . .) *ctx*), *mrk*]] = (**Stx** (**List** mark[[*stx*, *mrk*]] . . .) (Mark *ctx mrk*))
rename[[(**Stx** *atom ctx*), *id*, *var*]] = (**Stx** *atom* (Rename *ctx id var* NULL))
rename[[(**Stx** (**List** *stx* . . .) *ctx*), *id*, *var*]] = (**Stx** (**List** rename[[*stx*, *id*, *var*]] . . .) (Rename *ctx id var* NULL))
defs[[(**Stx** *atom ctx*), σ]] = (**Stx** *atom* (Defs *ctx* σ))
defs[[(**Stx** (**List** *stx* . . .) *ctx*), σ]] = (**Stx** (**List** defs[[*stx*, σ]] . . .) (Defs *ctx* σ))

marksof[[(**Stx** *val* •)]] = ()
marksof[[(**Stx** *val* (Mark *ctx mrk*))]] = *mrk* $\oplus$ marksof[[(**Stx** *val ctx*)]]
marksof[[(**Stx** *val* (Rename *ctx id var* σ))]] = marksof[[(**Stx** *val ctx*)]]
marksof[[(**Stx** *val* (Defs *ctx* σ))]] = marksof[[(**Stx** *val ctx*)]]
$mrk_1 \oplus (mrk_1\ mrk_2\ \ldots)$ = $(mrk_2\ \ldots)$
$mrk_1 \oplus (mrk_2\ \ldots)$ = $(mrk_1\ mrk_2\ \ldots)$

strip[[(**Stx** *atom ctx*)]] = *atom*
strip[[(**Stx** (**List** *stx* . . .) *ctx*)]] = (**List** strip[[*stx*]] . . .)

nostops[[ξ]] = {$var \rightarrow transformer$ | $\xi(var) = transformer$
 and $transformer \neq$ Stop}

Figure 4: Meta-functions